

Series: Hardening Apache HTTPD in Docker (Part 2 of 4)

Author: H. Brister Wooley

Published: 2026-01-22

Series: Hardening Apache HTTPD in Docker (2/4)

Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-01-22-hardening-apache-docker-part2.md>

Last time we built a container that runs as non-root, binds to one IP, and has a read-only filesystem. Good start – but the web server itself still talks to attackers with default settings.

Here's the uncomfortable truth: a non-root container with default `httpd.conf` is just a slightly less dangerous liability. The attacker still gets directory listings, verb tampering (PUT/DELETE/TRACE), and version headers that map directly to CVEs.

Today we fix that. We configure the part that actually talks to attackers – the `httpd.conf` – with a philosophy I call **default deny, explicit allow**. It's not sexy, but it's the difference between "we detected an attack" and "the attack bounced off a 403."

httpd.conf: Default Deny, Explicit Allow

```
<Directory />
    Options FollowSymLinks
    Options -Indexes -Includes
    AllowOverride none
    Require all denied
</Directory>

<Directory "/var/www">
    <LimitExcept GET POST HEAD>
        deny from all
    </LimitExcept>
    AllowOverride None
    Require all granted
</Directory>

<Directory "/var/www/html">
    <LimitExcept GET POST HEAD>
        deny from all
```

```

    </LimitExcept>
    Options -Indexes +FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>

```

Three tiers, one philosophy: **default deny**.

Tier	Path	What's Allowed	What's Blocked
1	/ (root)	Nothing. Require all denied	Everything. No listing, no includes, no overrides, no access
2	/var/www	GET, POST, HEAD only	PUT, DELETE, TRACE, OPTIONS, PATCH, CONNECT
3	/var/www/html	Same methods + FollowSymLinks	Same verb restrictions

Why Each Directive Matters

Directive	Attack It Stops
Options -Indexes	Directory listing – attackers love finding backup.zip, .git/, config.php.bak
Options -Includes	Server-Side Includes injection – <!--#exec cmd="cat /etc/passwd" -->
AllowOverride None	.htaccess abuse – prevents rogue configs in uploaded directories from overriding your rules
<LimitExcept GET POST HEAD>	Verb tampering – PUT uploads webshells, DELETE removes files, TRACE enables XST
Require all denied (root)	Path traversal – even if someone bypasses the app, the filesystem root is locked

Most apps need GET (pages), POST (forms), HEAD (health checks). Nothing else. If your app needs PUT for uploads, add it *only* for that specific Location – not globally.

File Access Control: The Safety Net for Developer Mistakes

```

<Files ".ht*">
    Require all denied
</Files>

```

```

<FilesMatch
"\.(bak|config|dist|fla|inc|ini|log|psd|sh|sql|swp|htaccess|htpasswd|env|conf
ig|.js|md|gitignore|gitattributes|lock|yaml|editorconfig|org|asp|aspx|java|php
|mp3|txt|jsp|xhtml|git)$|~$">

```

```
Require all denied
</FilesMatch>
```

Two layers:

1. **<Files ".ht*">** – Blocks .htaccess, .htpasswd, .htgroups. Apache's own control files. If they somehow end up in the web root, they're not served.
2. **<FilesMatch> with 30+ extensions** – Blocks backup files (.bak, ~), config files (.config, .ini, .yaml), source code (.php, .java, .asp, .js), secrets (.env, .htpasswd), git metadata (.gitignore, .gitattributes), editor temp files (.swp, .lock).

Why it matters: Developers make mistakes. A .env with database creds gets committed. A backup.sql with production data gets left in /var/www/html. A .git folder gets deployed. This rule is your safety net – Apache simply refuses to serve them, returning 403. The request never reaches your application code.

I've seen this catch a .env.production file that a CI/CD pipeline accidentally copied to the web root. The developer swore they'd cleaned it up. Apache disagreed.

Server Information Leakage: Shut Up

```
ServerTokens Prod
ServerSignature Off
Header unset X-Powered-By
Header unset ETag
FileETag None
HostnameLookups off
UseCanonicalName Off
TraceEnable off
```

Every header you send is intel for an attacker. Send the minimum.

Directive	What It Stops	Why It Matters
ServerTokens Prod	Header becomes Server: Apache – no version, no OS	Attackers fingerprint versions to match CVEs. Apache/2.4.62 (Ubuntu) tells them exactly what exploits to try
ServerSignature Off	No footer on 404/500 pages with version/build info	Error pages leak the same info. Turn it off
Header unset X-Powered-By	Strips PHP, Servlet, or framework fingerprints	X-Powered-By: PHP/8.2.1 – more version intel
FileETag None	Prevents inode leakage via ETag headers	Default ETag includes inode number. Inode numbers help

Directive	What It Stops	Why It Matters
		attackers map filesystem structure
HostnameLookups off	Disables reverse DNS lookups for logging	Each request triggers a PTR query. Slow, fills logs with hostnames you don't need
UseCanonicalName Off	Stops Apache from constructing self-referential URLs with ServerName	Prevents host header injection from poisoning redirects
TraceEnable off	Blocks TRACE method (cross-site tracing vector)	TRACE echoes the request back – steals HttpOnly cookies via XST

What This Means for Your Strategy

Here's the strategic shift: most teams treat `httpd.conf` as "configuration." It's not. It's **policy enforcement**. Every `Options -Indexes` is a decision: "we don't serve directory listings." Every `LimitExcept GET POST HEAD` is a contract: "this API only accepts these verbs."

When you frame it as policy, two things happen: 1. **Auditors stop asking questions** – the config is the documentation 2. **Developers stop fighting you** – they understand the *why*, not just the *what*

And that `.env` file catch? That's not luck. That's defense in depth doing its job – the layer you forgot about saves you when the layer you relied on fails.

What's Coming Next

Part 3 covers HTTP security headers (HSTS, CSP, Permissions-Policy), TLS hardening (why TLS 1.2 only, why no CBC), and the ModSecurity WAF config with surgical rule exclusions.

Plus the one CSP mistake that breaks your login page and how to fix it without 'unsafe-inline'.

[Read Part 3 ->](#)

Appendix: CompTIA Security+ SY0-701 CEU Mapping

This appendix maps each technique in this post to CompTIA Security+ exam objectives for CEU submission.

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
Default-deny Directory structure	Domain 3: Security Architecture	3.2: Implement secure system design	Least privilege filesystem access control
Options -Indexes	Domain 3: Security Architecture	3.2: Implement secure system design	Prevents information disclosure via directory listing
Options -Includes	Domain 2: Security Architecture	2.3: Implement application security	Blocks Server-Side Includes injection (code execution)
AllowOverride None	Domain 3: Security Architecture	3.2: Implement secure system design	Prevents configuration override via .htaccess
<LimitExcept GET POST HEAD>	Domain 3: Security Architecture	3.4: Implement secure network architecture	HTTP method restriction prevents verb tampering attacks
FilesMatch blocking 30+ sensitive extensions	Domain 2: Security Architecture	2.3: Implement application security	Prevents accidental exposure of secrets, source, backups
ServerTokens Prod + ServerSignature Off	Domain 3: Security Architecture	3.2: Implement secure system design	Reduces attack

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
Header unset X-Powered-By	Domain 3: Security Architecture	3.2: Implement secure system design	surface by minimizing version disclosure Removes technology fingerprinting vectors
FileETag None	Domain 3: Security Architecture	3.2: Implement secure system design	Prevents inode enumeration via ETag headers
TraceEnable off	Domain 2: Security Architecture	2.3: Implement application security	Blocks cross-site tracing (XST) attack vector
UseCanonicalName Off	Domain 3: Security Architecture	3.4: Implement secure network architecture	Prevents host header injection poisoning redirects

Primary Domain Coverage: Domain 3 (Architecture & Engineering) – ~60% | Domain 2 (Architecture) – ~40%

Total Security+ Mapping: >=50% of content maps to SY0-701 objectives